

基于 Coq 的微内核操作系统程序验证方法研究

张忠秋, 董云卫, 张 雨, 张 凡

(西北工业大学 计算机学院, 陕西 西安 710072)

摘要: 机载嵌入式程序的可信属性验证是新一代飞机研制最关注的软件质量保障问题; 基于定理证明的程序形式化验证方法是一种可靠和严格的软件正确性验证技术; 文中在深入分析微内核操作系统的基础上, 应用霍尔逻辑针对机载嵌入式软件核心代码开展程序验证技术研究, 根据霍尔逻辑的相关推理规则进行程序验证, 并在定理证明辅助工具 Coq 中形式化表示霍尔逻辑的推理规则, 针对机载操作系统的部分程序代码实例进行验证; 实验结果表明基于定理证明的程序验证方法可以对软件程序代码的正确性进行验证, 从而帮助软件提供商开发高可信的机载嵌入式软件。

关键词: 程序验证; 霍尔逻辑; 推理系统; 定理证明

Research on Microkernel Operating System Program Verification Based on Coq

Zhang Zhongqiu, Dong Yunwei, Zhang Yu, Zhang Fan

(College of Computer Science, Northwestern Polytechnical University, Xi'an 710072 China)

Abstract: The credibility verification for airborne embedded software is a software quality problem which is drawing more and more attention. Program verification based on theorem proving is a reliable and rigorous method. Combined with the analysis of microkernel operating system, the paper takes Hoare logic as rationale to carry out the study of verification for airborne embedded software core code to verify programs. Based on Hoare logic we give the formal description of some inference rules in Coq, then give a case study to the verification of real-world systems code derived from airborne operating system. The result shows that theorem proving approach can complete program verification and provide high degree of assurance of airborne embedded software.

Key words: program verification; Hoare logic; inference system; theorem proving

0 引言

随着嵌入式设备的广泛应用, 嵌入式系统的设计和结构也变得越来越复杂, 因此, 嵌入式操作系统就必不可少。通常, 嵌入式微内核操作系统占用空间小, 执行效率高, 主要提供一组最基本的服务, 如进程调度、进程间通信、存储管理、处理 I/O 设备等。由于微内核具有很好的扩展性及良好的层次化机构, 并且可以简化应用程序的开发, 现代的航空机载系统也采取了微内核设计结构, 以提高各种数据的处理速度^[1], 从而提高信息的处理能力并针对突发情况及时采取应急措施。例如, 新一代飞机的综合化航空电子嵌入式软件系统的程序代码规模多达到 300 万行代码以上, 其中, 系统运行所依赖的核心部分及机载微内核操作系统内核代码只有 7000 多行, 保证这些核心代码的正确性直接影响了综合化航空电子嵌入式软件系统及其相关机载设备能否准确完成设计要求的各项功能。

正确性是程序的重要属性之一, 也是程序正确性验证是可信软件的基本要求, 因此长期以来, 如何保证程序的正确性, 提高软件的可信度一直是机载嵌入式软件质量保障的重要课题。目前, 软件测试只能检测出软件产品中层出不穷的错误, 不能实现对机载软件完全的正确性验证。在 airbus、Boeing 等飞机制造公司的支持下, 软件设计研究人员利用形式化方法开

展软件正确性的研究。

程序正确性形式化验证是指用形式化方法描述程序的规范, 然后通过一定的验证规则对这些形式化规范以及相关代码进行验证, 判断这个程序是否依照程序规范所指示的方式执行。程序正确性验证通常采用测试、模型检测、逻辑推理验证等方法。其中, 基于逻辑推理的验证方法将程序转换为逻辑推理问题, 霍尔逻辑是逻辑推理验证方法中应用广泛的验证逻辑。本文主要是利用基于逻辑推理的验证方法, 采用霍尔逻辑对微内核操作系统程序验证方法进行探索和研究。

1 定理证明辅助工具 Coq

Coq^[1] 是一种基于高阶逻辑的计算机辅助定理证明工具, 是由法国国家计算机科学与控制研究所 (INRIA) 的研究人员开发的, 采用 Objective Caml 语言实现, 并基于元逻辑框架 CiC (Calculus of inductive Constructors) 的逻辑证明工具, CiC 是一个带有丰富类型系统的类型化 lambda 演算。

1.1 Coq 系统组成

Coq 系统主要由证明开发系统和证明检查器两部分组成^[2]。证明开发系统是一个证明环境, 帮助用户开发证明。证明检查器是一个用来验证被形式化表示的程序的程序。证明检查器的核心是类型检查算法, 这个算法通过检查程序是否满足规范来判定证明是否正确。证明检查包含了数学理论的自动验证和定义、公理、证明的自动验证。Coq 为用户提供了交互模式和编译模式两种模式, 交互模式解释用户从键盘上输入的命令类型, 也可以作为调试模式, 用户可以开发自己的定理并进行证明; 编译模式执行从文件中读取的命令, 也可以作为证明检查器, 通过文件得到定理, 从而确定其正确性。

1.2 在 Coq 中验证程序的原理

在 Coq 中, 通过输入策略来构造相应的证明, 策略实现

收稿日期: 2011-01-10; 修回日期: 2011-02-25。

基金项目: 国家自然科学基金项目 (60736017); 航空科学基金项目 (20081953012)。

作者简介: 张忠秋 (1981-), 女, 硕士研究生, 主要从事程序验证方法研究。

董云卫, 男, 教授, 主要从事嵌入式软件设计与验证方法研究。

后反向推理,即当应用某个策略到目标上时,系统会产生相对应的子目标,每一个子目标都是用数字标识的,通过对各个子目标的证明来完成整个证明过程。证明过程中,在应用策略到给定目标时,系统会检查一些应用该策略必须满足的前提条件,如果这个条件不能被满足,则该项策略的应用就会失败,系统给出相应的错误提示信息。同时,由于 Coq 提供了丰富的策略库,对同一个目标可以采用不同的策略或者策略组合来完成证明,因此策略的选择对证明的过程也是非常重要的。Coq 具有强大的开发功能,在这个系统中,可以形式化定义自己的逻辑系统、推理系统等,证明开发系统采用与用户交互的方式构造证明,它支持策略和证明检查器的使用,降低了证明的复杂度,一定程度上实现了证明的自动化程度,提高了证明的效率。Coq 系统中有很多证明策略供用户选择帮助用户构造证明,因此 Coq 是一个交互式系统,用户可以把一个难处理的证明分解成一系列引理,也可以根据需要选择适合于此难题的策略来证明。Coq 采用反向推理的方法构造证明,即给定目标后,根据输入的策略分解当前目标得到一些简单的子目标,然后通过构造子目标的证明得到整个目标的证明,最后通过系统提供的证明检查器检查证明的正确性。

1.3 基于 Coq 的程序验证

在推理和编程方面,Coq 都拥有足够强大的表达能力,因此 Coq 已经被广泛用于程序代码的证明和验证相关的研究,如程序分析框架,程序验证等等。基于 Coq 构造程序证明的过程可分为标注阶段、形式化阶段和证明阶段三个阶段,如图 1 所示。



图 1 Coq 程序证明流程图

其中,标注阶段完成对程序段的前条件和后条件、循环不变式、要证明的程序属性的描述,其中前条件是程序段的初始状态,包括参数类型、取值等,后条件是程序段正确执行完后的状态;形式化阶段在 Coq 系统中采用形式化的方法完成对程序、标注的相关定义;证明阶段根据形式化的程序、标注来抽取关于程序满足程序属性的定理,然后在 Coq 系统中对定理进行证明。

2 基于霍尔逻辑的程序验证

2.1 霍尔逻辑的基本思想

霍尔逻辑^[3]是广泛应用的程序验证逻辑系统,用于对命令式语言程序进行推理验证。其基本思想是:在代码及其调用者之间构建一种合同式的规范说明,由一个前置条件和一个后置条件构成。前置条件和后置条件均是一个断言,用来说明程序的规范。前置条件描述代码执行前程序状态必须满足的条件,主要包括参数的取值范围、数据结构等;后置条件描述代码正确运行后程序状态所需要满足的条件,主要包括程序执行后的参数

相关信息等。

用霍尔逻辑对程序进行验证时,程序正确性表示为 Hoare 三元组 $\{P\}S\{Q\}$,其中 P 是前置条件, S 是程序段, Q 是后置条件。 $\{P\}S\{Q\}$ 表示如果 P 在执行程序 S 之前为真,那么 Q 在程序 S 执行之后为真。例如: $\{X < -4\}Y = X + 1\{Y < 1\}$,这个例子表明以任何满足 $X < -4$ 的 X 开始执行程序 $Y = X + 1$,则程序必终止,并且终止时 Y 的值满足 $Y < 1$ 。程序的正确性是相对于规范的,即 S 满足了 $\langle Q, R \rangle$ 的要求, S 相对于 $\langle Q, R \rangle$ 是正确的。

2.2 霍尔逻辑的基本推理规则

霍尔逻辑基于程序中插入的断言进行程序正确性的形式化证明,提供了一种验证程序正确性的基础,对高级语言中的基本控制结构,如条件语句、循环语句等都有相应的推理规则。要证明程序的正确性,就需要不断地使用这些推理规则进行推导。这些规则主要有:

$$\text{skip}; \{P\} \text{skip} \{P\} \quad (1)$$

$$\text{assign}; \{P[x \mapsto a]\} x := a \{P\} \quad (2)$$

$$\text{sequence}; \frac{\{P\}S_1\{Q\}, \{Q\}S_2\{R\}}{\{P\}S_1; S_2\{R\}} \quad (3)$$

$$\text{if}; \frac{\{b \wedge P\}S_1\{Q\}, \{\neg b \wedge P\}S_2\{Q\}}{\{P\}\text{if}(b)\text{then}(S_1)\text{else}(S_2)\{Q\}} \quad (4)$$

$$\text{while}; \frac{\{b \wedge P\}S\{P\}}{\{P\}\text{while}(b)\text{do}(S)\{\neg b \wedge P\}} \quad (5)$$

$$\text{cons}; \frac{P \rightarrow P_1, \{P_1\}S\{Q_1\}, Q \rightarrow Q_1}{\{P\}S\{Q\}} \quad (6)$$

公式 (1) 是空语句法则,表示 P 在执行空语句之前为真,那么在执行空语句之后其依然为真,空语句的执行不改变 P 的值。

公式 (2) 表示赋值公理, $P[x \mapsto a]$ 表示把 P 中出现的 x 替换成 a 得到的谓词。赋值公理表示如果执行赋值语句 $x = a$ 后 P 为真,则在执行赋值语句之前 $P[x \mapsto a]$ 为真,其中 x 为自由变元。

公式 (3) 是复合规则,表示 $\{P\}S_1; S_2\{R\}$ 的成立需要以 $\{P\}S_1\{Q\}$ 和 $\{Q\}S_2\{R\}$ 的成立为前提条件。

公式 (4) 是分支法则,表示如果执行 S_1 ,那么执行前 P 和 b 为真,如果执行 S_2 ,那么执行前 P 为真, b 为假。

公式 (5) 是循环法则,表示 P 和 b 为真时执行 S , S 执行完后, b 为假而 P 为真, P 是循环中必须保持为真的循环不变式^[4]。

公式 (6) 是后继法则,是一条表示因果关系的法则,表示如果一个 Hoare 三元组为真,那么把前置条件加强或者后置条件减弱也为真。

当用程序逻辑验证一个程序的正确性时,程序的规范说明要按前面的 Hoare 逻辑三元组的形式 $\{P\}S\{Q\}$ 给出,当前置条件 Q 满足时,正确的程序必须保证其程序状态和变量在经过程序执行后满足其后置条件。因此一个程序的正确性是与给定的前置条件和后置条件相关的。由于整个程序验证过程中的每一步都要使用程序逻辑中的公理和推理规则,所以程序正确性的验证过程实际上是一个用程序逻辑构造定理证明的过程。

2.3 霍尔逻辑验证实例分析

本节基于微内核操作系统 ucos-1^[5] 给出一个简化的两个进程 A、B 共享资源问题在霍尔逻辑推理系统下的分析。设进程 A、B 共享资源 p, q , 进程 A 或 B 必须同时占有资源 p, q 才

能得以运行, 即要求 p 、 q 的状态保持一致。用 i ($i=0, 1, 2$) 表示资源的状态, $i=0$ 表示资源空闲, $i=1$ 表示资源被进程 A 占用, $i=2$ 表示资源被进程 B 占用。用 $lock$ 表示进程对资源的占用与释放, $lock=false$ 表示进程释放占用的资源, $lock=true$ 表示进程正在占用资源, 进程代码段如图 2 (a) 所示。以 Hoare 逻辑证明程序的过程如图 2 (b) 所示, 其中, 函数入口前置条件为 $\{true\}$ 表明该进程总是可以安全的被调用, 斜体所示为程序的规范断言, 进程 B 的过程类似进程 A, 在此不作赘述。由上述推理过程可以看出, 该进程在资源 p 、 q 状态保持一致 (即该过程的循环不变式) 的约束下正确地运行, 不会出现 A 和 B 各占一个资源而不能同时得到两个资源的情况, 即不会出现进程 A 等待进程 B 释放资源而 B 也在等待 A 释放资源, 从而出现死锁的现象, 保证了程序的正确执行。

```

.....
while(true)
{lock=false;
  atomic
  {if(p==0)
  { p=1;
    q=1;
    lock=true;
  }}//end if
  }//end atomic
if(lock)
{ /* using resource to do something */
  atomic
  { p=0;
    q=0;
  }}//end atomic
  /* do something else */
} //end if
} //end while
.....

```

图 2 (a) 代码段

```

{true}
while (true)
{
{true}
lock=false
{lock=false}//assign rule
atomic
{
{lock=false ∧ p=q}//skip rule
if(p=0)
{
{lock=false ∧ p=q=0}//if rule
p=1
{lock=false ∧ p=l ∧ q=0}//assign rule
q=1
{lock=false ∧ p=q=l}//assign rule
lock=true
} //end if
}
}

```

```

{lock=true ∧ p=q=l ∨ lock=false ∧ p=q=0}//if rule
} //end atomic
{lock=true ∧ p=q=l ∨ lock=false ∧ p=q=0}//slap rule
if(lock)
{
{lock=true ∧ p=l}//cons rule (strong pre-condition)
/* use resource to do something */
{lock=true ∧ p=q=l}//skip rule
atomic
{
{lock=true ∧ p=q=l}//skip rule
p=0
{lock=true ∧ p=0 ∧ q=l}//assign rule
q=0
{lock=true ∧ p=q=0}//assign rule
} //end atomic
{lock=true ∧ p=q=0}//skip rule
/* to do something else */
} //end if
{lock=true ∧ p=q=0 ∨ lock=false ∧ p=q=0}//if rule
} //end while

```

图 2 (b) 霍尔逻辑推理进程图

2.4 推理规则在 Coq 中的表示及实例验证

推理规则构成了逻辑推理系统的基础, 要在交互式定理证明工具 Coq 中进行程序证明, 首先要在 Coq 中定义推理规则, 搭建验证环境即霍尔逻辑推理系统^[6]。Hoare 逻辑三元组在 Coq 中用谓词 $correct\ s\ P\ Q$ 来表示, 图 3 给出了 2.2 节各推理规则在 Coq 中的形式化表示。

- (1) forall (P : Eprop), correct skip P P
- (2) forall v e (P Q : Eprop), (forall E, P E -> Q (upd E v (eval E e))) -> correct (assign v e) P Q
- (3) forall s1 s2 P P Q, correct s1 P P -> correct s2 P Q -> correct (seq s1 s2) P Q
- (4) forall e s I, correct s (fun E => I E ∧ eval E e <> 0) I -> correct (while e s) I (fun E => I E ∧ eval E e = 0)
- (5) forall e s1 s2 P Q, correct s1 (fun E => P E ∧ eval E e < 0) Q -> correct s2 (fun E => P E ∧ eval E e = 0) Q -> correct (br e s1 s2) P Q
- (6) forall s (P P Q Q' : Eprop), (forall E, P E -> P E) -> (forall E, Q E -> Q E) -> correct s P Q' -> correct s P Q.

图 3 推理规则在 Coq 中的表示

基于以上分析, 对 2.3 节给出的实例的 if 分支语句段的形式化表示如图 4 所示。接着应用 Hoare 逻辑推理系统推理规则以及 Coq 系统的策略组合对图 4 中的定理 Theorem resource_ok 进行证明。例如利用 apply okBr 策略后由系统自动生成两个子目标, 如图 5 所示。应用赋值规则和 unfold resource_pre 等策略将生成的第一个子目标展开, 对展开的各个子目标选择合适的策略分别解决, 然后应用 unfold resource_post、simply 等策略对第二个子目标进行证明, 重复此过程来证明每一步生成的子目标。当所有目标都得到证明时, 定理 Theorem resource_ok 便证明成功。该证明过程表明基于定理证明的程序验证方法能够正确地完成程序验证过程, 在程序验证方面有一

定的实用价值。

```

Definition resource_pre(x y z:Var):Eprop:=
  fun E=>E x=0 ∧ E y=0 ∧ E z=0.
Definition resource_post(x y z:Var):Eprop:=
  fun E=>E x=1 ∧ E y=1 ∧ E z=1
  'E x=0 ∧ E y=0 ∧ E z=0.
Definition resource_prog(x y z:Var):=
  br( C(equal_0 y))
  (x <- (C 1); y <- (C 1); z <- (C 1))
  nil.
Theorem resource_ok;
  correct (resource_prog VX VY VZ)
  (resource_pre VX VY VZ)
  (resource_post VX VY VZ)
Proof.

```

图 4 程序形式化表示

```

2 subgoals
_____ (1/2)
correct (VX<-C 1;;VY<-C 1;;VZ<-C 1)
  fue E:Env=>resource_pre VX VY VZ E ∧
  eval E (C(equal_0 VY))<=>0)
  (resource_post VX VY VZ)
_____ (2/2)
correct nil
  (fun E:Env=>resource_pre VX VY VZ E ∧
  (eval E (C(equal_0 VY))=0
  (resource_post VX VY VZ)

```

图 5 子目标示意图

3 逻辑推理相关工作讨论

从理论上来说,基于逻辑推理的证明方法是非常强大的,能证明各种程序的正确性,但是,往往要求用户具有很高的水平,比如能给出程序中的循环不变式,另外还要求有表达能力非常强的定理证明器。而从目前的研究进展来看,这些定理证明器很难自动化或者自动化程度不高,开发辅助用户将断言谓词自动转换到定理证明器中的自动化工具和提高定理证明器的自动化验证程度将是一个挑战性非常大的研究课题。

霍尔逻辑是目前使用广泛的证明命令式语言程序性质的经典方法,虽然可以很容易地扩展它,使他能够确定一些较简单

的别名关系,但很难处理别名关系复杂的指针程序。为了研究指针程序的验证^[7],出现了类 C 小语言 PointerC,其基本安全策略要求程序运行时不出现通过 NULL 指针或者悬空指针来存取所指向的操作,不把这两类指针作为 free 函数调用的实在参数,不发生内存泄露等。为此,要为 PointerC 语言设计一种指针逻辑,其目的是将精确的指针分析应用于程序验证过程。

霍尔逻辑在处理包含可操作数据结构程序的验证时也是很困难的,因此出现了分离逻辑^[8]的研究。分离逻辑通过提供表达显示分析的逻辑连接词以及相应的推导规则,消除了共享的可能,能够以自然的方式来描述计算过程中内存的属性和相关操作,从而简化了对指针程序的验证工作。分离逻辑能够支持局部推理,然后通过相关的推导规则扩展到全局状态,支持对并发程序和资源管理的验证,因此分离逻辑是验证程序可信性的一种重要方法。

4 结论

本文介绍了基于霍尔逻辑推理系统开展机载嵌入式操作系统核心代码程序验证方法,并利用交互式定理证明辅助工具 Coq 给出了相关推理规则的形式化表示和实现以及相关定理的证明。同时给出使用基于霍尔逻辑推理的证明方法对两个进程共享资源问题的推理过程,并在 Coq 中实现了简单验证,为采用定理证明方法对程序验证提供了理论证明依据,为操作系统程序的验证奠定了良好的基础。

参考文献:

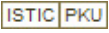
- [1] The Coq development team. The Coq Proof Assistant Reference Manual [EB], Version V8.0, <http://coq.inria.fr/>, 2005.
- [2] Yves Bertot, Pierre Casteran. Coq's Art: The Calculus of Inductive Constructions [M]. Berlin: Springer-Verlag, 2004.
- [3] C A R Hoare. An axiomatic basis for computer programming [J]. Communications of the ACM, 1969, 12 (10): 576-580.
- [4] David Gries. The Science of Programming [M]. Springer, 1981.
- [5] Jean J. Labrosse. MicroC/OS-II The Real-Time Kernel Second Edition [M]. CMP Books, 2005. 6.
- [6] Xinyu Jiang, Yu Guo, Yiyun Chen. The Logical Approach to Low-level Stack Reasoning [A]. 3th IEEE International Symposium on Theoretical Aspects of Software Engineering [C]. 2009.
- [7] 陈意云,李兆鹏,王志芳,华保健.一种用于指针程序验证的指针逻辑 [J]. 软件学报, 2009, 15 (1): 1-13.
- [8] 黄达明,曾庆凯.基于分离逻辑的程序验证技术研究 [J]. 软件学报, 2009, 20 (8): 2051-2060.

(上接 1938 页)

- [4] Takahisa Kobayashi, Donald L. Simon. Application of a Bank of Kalman Filters for Aircraft Engine Fault Diagnostics [R]. NASA/TM-2003-212526.
- [5] Xue W, Guo Y Q, Zhang X D. Application of a bank of Kalman filters and a robust Kalman filter for aircraft engine sensor/actuator fault diagnosis [J]. International Journal of Innovative Computing, Information and Control, 2008, 4 (12): 353-361.
- [6] 陈 恬,孙健国,郝 英.基于神经网络和证据融合理论的航空

发动机气路故障诊断 [J]. 航空学报, 2006, 27 (6): 1015-1017.

- [7] 鲁 峰,黄金泉,陈 煜.航空发动机部件性能故障融合诊断方法研究 [J]. 航空动力学报, 2009, 24 (7): 1650-1653.
- [8] Rogova G. Combining the results of several neural network classifiers [J]. Neural Networks, Networks, 1994, 7 (5): 777-781.
- [9] 李 睿,郭迎清,吴文斐.航空发动机传感器故障诊断设计与验证综合仿真平台 [J]. 计算机测量与控制, 2010, 18 (3): 527-529.

作者：[张忠秋](#)，[董云卫](#)，[张雨](#)，[张凡](#)，[Zhang Zhongqiu](#)，[Dong Yunwei](#)，[Zhang Yu](#)，[Zhang Fan](#)
作者单位：[西北工业大学计算机学院, 陕西西安, 710072](#)
刊名：[计算机测量与控制](#) 
英文刊名：[Computer Measurement & Control](#)
年，卷(期)：2011, 19(8)

参考文献(8条)

1. [Jean J. Labrosse](#) [MicroC/OS-II The Real-Time Kernel Second Edition](#) 2005
2. [David Gries](#) [The Science of Programming](#) 1981
3. [C A R Hoare](#) [An axiomatic basis for computer programming](#) 1969(10)
4. [Yves Bertot; Pierre Casteran](#) [Coq' s Art: The Calculus of Inductive Constructions](#) 2004
5. [The Coq development team](#) [The Coq Proof Assistant Reference Manual. Version V8.0](#) 2005
6. [黄达明; 曾庆凯](#) [基于分离逻辑的程序验证技术研究](#) 2009(08)
7. [陈意云; 李兆鹏; 王志芳; 华保健](#) [一种用于指针程序验证的指针逻辑](#) 2009(01)
8. [Xinyu Jiang; Yu Guo; Yiyun Chen](#) [The Logical Approach to Low-level Stack Reasoning](#) 2009

本文链接：http://d.g.wanfangdata.com.cn/Periodical_jsjzdclykz201108040.aspx